6901012610196 Chonpatchut Boonpalit Cpr.E S.2

รายงาน Lab 7

ข้อที่ 1 การหาค่าสถิติจาก Array 2 มิติ

โปรแกรมรับจำนวนแถวและจำนวนคอลัมน์จากผู้ใช้ จากนั้นรับค่าตัวเลขแต่ละตำแหน่งใน Array 2 มิติ และนำข้อมูลไปใช้กับฟังก์ชันต่าง ๆ

1.1 ฟังก์ชัน find_max()

ฟังก์ชัน `find_max(a)` ใช้สำหรับหาค่าที่มากที่สุดใน Array 2 มิติ โดยกำหนดค่าแรก `a[0][0]` ให้เป็นค่ามากที่สุดในตอนเริ่มต้น จากนั้นใช้ลูปตรวจสอบค่าทุกตำแหน่ง ถ้าพบค่าที่มากกว่า `maximum` จะเปลี่ยนค่า `maximum` เป็นค่านั้น แล้วคืนค่ากลับมา

ผลลัพธ์จะแสดงในรูปแบบ

`Max = ค่าที่มากที่สุด`

1.2 ฟังก์ชัน find_min()

ฟังก์ชัน `find_min(a)` ใช้สำหรับหาค่าที่น้อยที่สุดใน Array 2 มิติ โดยเริ่มจากกำหนด `a[0][0]` เป็นค่าต่ำสุด จากนั้นตรวจสอบสมาชิกทุกตำแหน่ง ถ้าพบค่าที่น้อยกว่า `minimum` จะเปลี่ยนค่าเป็นค่านั้น

ผลลัพธ์จะแสดงในรูปแบบ

`Min = ค่าที่น้อยที่สุด`

1.3 ฟังก์ชัน find_average()

ฟังก์ชัน `find_average(a)` ใช้สำหรับหาค่าเฉลี่ยของข้อมูลทั้งหมดใน Array 2 มิติ โดยใช้ตัวแปร `total` สำหรับเก็บผลรวม และ `count` สำหรับนับจำนวนข้อมูล เมื่อวนตรวจสอบข้อมูลครบทั้งหมดแล้ว จะนำ `total` หารด้วย `count`

ผลลัพธ์จะแสดงในรูปแบบ

`Average = ค่าเฉลี่ย`

1.4 ฟังก์ชัน is_found()

ฟังก์ชัน `is_found(a, value)` ใช้ตรวจสอบว่าค่าที่ผู้ใช้ต้องการค้นหามีอยู่ใน Array หรือไม่ ถ้าพบค่าที่ตรงกันจะคืนค่า `True` ทันที แต่ถ้าตรวจสอบครบทุกตำแหน่งแล้วไม่พบ จะคืนค่า `False`

ผู้ใช้สามารถป้อนค่าที่ต้องการค้นหาได้จาก

```
Enter value to find:
```

และโปรแกรมจะแสดงผลเป็น `Found = True` หรือ `Found = False`

1.5 การบวก Matrix

โปรแกรมกำหนด Matrix `a` และ `b` เป็น

```
a = [[1, 2], [3, 4]]
```

```
b = [[5, 6], [7, 8]]
```

การบวก Matrix จะนำสมาชิกที่อยู่ตำแหน่งเดียวกันมาบวกกัน เช่น `a[0][0] + b[0][0]` จากนั้นเก็บผลลัพธ์ลงใน Matrix ใหม่ชื่อ `add`

ผลลัพธ์คือ

```
Add = [[6, 8], [10, 12]]
```

1.6 การลบ Matrix

การลบ Matrix ทำงานคล้ายกับการบวก แต่เปลี่ยนเป็นการนำสมาชิกตำแหน่งเดียวกันมาลบกัน

ผลลัพธ์คือ

```
Subtract = [[-4, -4], [-4, -4]]
```

1.7 การคูณ Matrix

การคูณ Matrix ใช้ลูป 3 ชั้นเพื่อคำนวณตามหลักการคูณ Matrix โดยนำสมาชิกในแถวของ Matrix `a` มาคูณกับสมาชิกในคอลัมน์ของ Matrix `b` แล้วนำผลลัพธ์มาบวกกัน

ผลลัพธ์คือ

```
Multiply = [[19, 22], [43, 50]]
```

Error ที่พบ

ในการเขียนโปรแกรมส่วนนี้ สิ่งที่ต้องระวังคือการกำหนดจำนวนแถวและคอลัมน์ให้ถูกต้อง เพราะถ้าขนาดของ Matrix ไม่เหมาะสม อาจทำให้เกิด `IndexError` หรือไม่สามารถคูณ Matrix ได้ตามหลักการ

นอกจากนี้ ถ้าไม่มีข้อมูลใน Array แล้วเรียก `a[0][0]` ในฟังก์ชัน `find_max()` หรือ `find_min()` จะเกิด `IndexError` ได้เช่นกัน

ข้อที่ 2 การเก็บและวิเคราะห์อุณหภูมิ 4 สัปดาห์

โปรแกรมใช้ Array 2 มิติ `weekly_temp` สำหรับเก็บข้อมูลอุณหภูมิ โดยมี 4 แถวแทน 4 สัปดาห์ และแต่ละแถวมีข้อมูล 7 วัน

ผู้ใช้สามารถเลือกสัปดาห์ที่ต้องการตรวจสอบโดยป้อนค่าตั้งแต่ 0 ถึง 3

2.1 ฟังก์ชัน week_average()

ฟังก์ชัน `week_average(weekly_temp, w)` ใช้หาค่าอุณหภูมิเฉลี่ยของสัปดาห์ที่เลือก โดยวนลูปทั้ง 7 วัน นำอุณหภูมิมาบวกกันในตัวแปร `total` แล้วหารด้วย 7

2.2 ฟังก์ชัน week_max()

ฟังก์ชัน `week_max(weekly_temp, w)` ใช้หาค่าอุณหภูมิสูงสุดของสัปดาห์ที่เลือก โดยเริ่มจากอุณหภูมิของวันแรก แล้วเปรียบเทียบกับวันถัดไปทีละวัน ถ้าพบค่าที่มากกว่า จะเปลี่ยนค่า `maximum`

2.3 ฟังก์ชัน week_min()

ฟังก์ชัน `week_min(weekly_temp, w)` ใช้หาค่าอุณหภูมิต่ำสุดของสัปดาห์ที่เลือก หลักการทำงานเหมือน `week_max()` แต่เปลี่ยนเป็นการหาค่าที่น้อยกว่า

2.4 ฟังก์ชัน overall_max()

ฟังก์ชัน `overall_max(weekly_temp)` ใช้หาค่าอุณหภูมิสูงสุดจากข้อมูลทั้ง 4 สัปดาห์ โดยเรียกใช้ `week_max()` ของแต่ละสัปดาห์ แล้วนำค่าที่ได้มาเปรียบเทียบกัน

การใช้ `week_max()` ซ้ำทำให้ไม่ต้องเขียนขั้นตอนการหาค่าสูงสุดของแต่ละวันใหม่อีกครั้ง

2.5 ฟังก์ชัน overall_min()

ฟังก์ชัน `overall_min(weekly_temp)` ใช้หาค่าอุณหภูมิต่ำสุดจากข้อมูลทั้งหมด โดยเรียกใช้ `week_min()` ของแต่ละสัปดาห์แล้วเปรียบเทียบค่าที่ได้

Error ที่พบ

ข้อที่ต้องระวังคือค่าของ `w` ต้องอยู่ในช่วง 0–3 เพราะข้อมูลมีเพียง 4 สัปดาห์ ถ้าผู้ใช้ป้อนค่าอื่น เช่น 4 โปรแกรมจะไม่สามารถเข้าถึง `weekly_temp[4]` ได้และจะเกิด `IndexError`

ดังนั้นข้อความ `Enter week (0-3):` จึงใช้บอกช่วงค่าที่สามารถป้อนได้อย่างชัดเจน

ข้อที่ 3 Treasure Hunt

โปรแกรมนี้เป็นเกมแบบ Text-mode ที่ให้ผู้เล่นควบคุมตัวละคร `P` เพื่อเดินไปหา Treasure `T` โดยใช้ปุ่ม W, A, S และ D

แผนที่ถูกเก็บในรูปแบบ Array 2 มิติ โดย `#` คือกำแพง `.` คือพื้นที่ที่สามารถเดินได้ `P` คือตำแหน่งผู้เล่น และ `T` คือตำแหน่งสมบัติ

3.1 ฟังก์ชัน show_map()

ฟังก์ชัน `show_map()` ใช้แสดงแผนที่และข้อมูลของเกมบนหน้าจอ

ฟังก์ชันจะใช้ลูปตรวจสอบแต่ละแถวและแต่ละคอลัมน์ของ `game_map` แล้วนำตัวอักษรแต่ละตัวมาต่อกันเป็นบรรทัดก่อนแสดงผล

นอกจากนี้ยังแสดงจำนวนก้าวของผู้เล่น และคำสั่งที่สามารถใช้ได้ เช่น W, A, S, D และ Q

3.2 ฟังก์ชัน move_player()

ฟังก์ชัน `move_player(direction)` ใช้จัดการการเคลื่อนที่ของผู้เล่นตามปุ่มที่ป้อนเข้ามา

ถ้าผู้เล่นกด

- `w` จะเดินขึ้น
- `s` จะเดินลง
- `a` จะเดินไปทางซ้าย
- `d` จะเดินไปทางขวา

โปรแกรมจะสร้างตำแหน่งใหม่ไว้ใน `new_row` และ `new_col` ก่อน แล้วตรวจสอบว่าตำแหน่งนั้นเป็นกำแพงหรือไม่

ถ้าเป็น `#` ผู้เล่นจะไม่สามารถเดินผ่านได้

ถ้าเป็น `T` โปรแกรมจะเปลี่ยนตำแหน่งของผู้เล่นและกำหนด `game_over = True` เพื่อจบเกม พร้อมแสดงจำนวนก้าวที่ใช้

ถ้าเป็นพื้นที่ว่าง โปรแกรมจะลบตำแหน่งเดิมของผู้เล่น เปลี่ยนตำแหน่งใหม่เป็น `P` และเพิ่มจำนวนก้าวขึ้น 1

3.3 ส่วนของ Main Program

โปรแกรมใช้

```
while game_over == False
```

เพื่อให้เกมทำงานต่อไปเรื่อย ๆ จนกว่าผู้เล่นจะเจอ Treasure หรือกด `q`

ถ้าผู้เล่นกด `q` โปรแกรมจะจบเกมทันที

ถ้าป้อนคำสั่งที่มีความยาวมากกว่า 1 ตัวอักษร โปรแกรมจะแจ้งให้ป้อนเพียงตัวอักษรเดียว

Error ที่พบ

สิ่งที่ต้องระวังคือการเดินออกนอกขอบเขตของ `game_map` เพราะโปรแกรมเข้าถึง `game_map[new_row][new_col]` โดยตรง หากตำแหน่งใหม่อยู่นอก Array อาจทำให้เกิด `IndexError`

ในแผนที่ที่กำหนดไว้ ปัญหานี้ถูกหลีกเลี่ยงด้วยการสร้างกำแพง `#` ล้อมรอบแผนที่ ทำให้ผู้เล่นไม่สามารถเดินออกจากขอบเขตได้

อีกส่วนที่ต้องระวังคือการใช้ `global` ในฟังก์ชัน `move_player()` เนื่องจากฟังก์ชันต้องแก้ไขค่าของ `player_row`, `player_col`, `steps` และ `game_over` ที่อยู่นอกฟังก์ชัน

ข้อที่ 4 เกม XO (Tic Tac Toe)

โปรแกรมเป็นเกม XO ขนาด 3×3 ผู้เล่นสลับกันลง X และ O โดยคลิกที่ช่องที่ต้องการลง ตัวแปร `board` ใช้เก็บสถานะของกระดาน ส่วน `turn` ใช้เก็บว่าถึงตาของ X หรือ O

4.1 ฟังก์ชัน setup()

ฟังก์ชัน `setup()` ใช้กำหนดขนาดหน้าต่างเกมเป็น 450×550 และกำหนดการจัดตำแหน่งของข้อความให้อยู่ตรงกลาง

4.2 ฟังก์ชัน draw()

ฟังก์ชัน `draw()` ใช้แสดงหน้าจอเกม โดยแสดงชื่อเกม สถานะของผู้เล่นว่าเป็นตาของ X หรือ O และเรียก `draw_board()` เพื่อแสดงกระดาน

ถ้าเกมจบแล้วจะแสดงข้อความ เช่น `X wins!` หรือ `Draw!` แทนข้อความ Turn

4.3 ฟังก์ชัน draw_board()

ฟังก์ชัน `draw_board()` ใช้สร้างกระดาน XO ขนาด 3×3 โดยใช้ลูป `while` 2 ชั้นในการวนแถวและคอลัมน์ แต่ละช่องมีขนาด 100×100 และนำค่าจาก `board[i][j]` มาแสดงในช่องนั้น

4.4 ฟังก์ชัน mousePressed()

ฟังก์ชัน `mousePressed()` ทำงานเมื่อผู้เล่นคลิกเมาส์ โดยคำนวณจากตำแหน่ง `mouseX` และ `mouseY` ว่าคลิกช่องไหน จากนั้นตรวจสอบว่าช่องนั้นว่างหรือไม่

ถ้าช่องว่าง โปรแกรมจะใส่ X หรือ O ลงไปตามค่าของ `turn` แล้วเรียก `check_win()` เพื่อตรวจสอบว่าชนะหรือไม่ ถ้าไม่มีผู้ชนะจะเรียก `check_draw()` เพื่อตรวจสอบว่ากระดานเต็มหรือไม่ ถ้ายังเล่นต่อได้จะเปลี่ยนตาจาก X เป็น O หรือ O เป็น X

4.5 ฟังก์ชัน check_win()

ฟังก์ชัน `check_win(player)` ใช้ตรวจสอบว่าผู้เล่นชนะหรือไม่ โดยตรวจสอบทั้งหมด 3 แบบ คือ

1. ตรวจสอบแนวนอนทั้ง 3 แถว
2. ตรวจสอบแนวตั้งทั้ง 3 คอลัมน์
3. ตรวจสอบแนวทแยง 2 เส้น

ถ้ามี X หรือ O เรียงกันครบ 3 ช่องในแนวใดแนวหนึ่ง จะคืนค่า `True`

4.6 ฟังก์ชัน check_draw()

ฟังก์ชัน `check_draw()` ใช้ตรวจสอบว่ากระดานเต็มหรือไม่ โดยวนดูทุกช่อง ถ้ายังมีช่องที่เป็น `""` อยู่ จะคืนค่า `False` แต่ถ้าทุกช่องถูกลงครบแล้วจะคืนค่า `True` และเกมจะแสดง `Draw!`

4.7 ฟังก์ชัน keyPressed()

ฟังก์ชัน `keyPressed()` ใช้สำหรับเริ่มเกมใหม่ เมื่อผู้เล่นกดปุ่ม R หรือ r โปรแกรมจะล้างข้อมูลใน `board` เปลี่ยน `turn` กลับเป็น X และตั้งค่า `game_over` เป็น `False`

Error ที่พบ

ปัญหาที่ต้องระวังคือการคำนวณตำแหน่งช่องจาก `mouseX` และ `mouseY` เพราะถ้าคลิกนอกพื้นที่กระดานอาจได้ค่าแถวหรือคอลัมน์ที่ไม่ถูกต้อง จึงต้องมีการตรวจสอบ `row` และ `col` ให้อยู่ในช่วง 0–2 ก่อนเข้าถึง `board[row][col]`

อีกปัญหาคือถ้าไม่ได้ใช้ `global` กับตัวแปร `turn`, `game_over` และ `message` ใน `mousePressed()` อาจไม่สามารถเปลี่ยนค่าตัวแปรที่อยู่นอกฟังก์ชันได้ จึงต้องประกาศ `global` ก่อนแก้ไขค่าเหล่านี้

ข้อที่ 5 เกม Minesweeper

โปรแกรมเป็นเกม Minesweeper ขนาด 8×8 มีระเบิดทั้งหมด 10 ลูก ผู้เล่นคลิกเปิดช่อง ถ้าเปิดเจอระเบิดจะจบเกม แต่ถ้าเปิดช่องที่ปลอดภัยทั้งหมดได้จะชนะ

ตัวแปร `mine` ใช้เก็บตำแหน่งระเบิด `number` ใช้เก็บจำนวนระเบิดรอบแต่ละช่อง และ `open_cell` ใช้เก็บสถานะว่าช่องนั้นถูกเปิดแล้วหรือยัง

5.1 ฟังก์ชัน setup()

ฟังก์ชัน `setup()` ใช้กำหนดขนาดหน้าต่างเกมเป็น 400×450 กำหนดการจัดข้อความให้อยู่ตรงกลาง และเรียก `new_game()` เพื่อสร้างเกมใหม่

5.2 ฟังก์ชัน draw()

ฟังก์ชัน `draw()` ใช้แสดงชื่อเกม จำนวนระเบิด และข้อความสถานะของเกม

ถ้าแพ้จะแสดง `GAME OVER - Press R to restart`

ถ้าชนะจะแสดง `YOU WIN! - Press R to restart`

จากนั้นเรียก `draw_board()` เพื่อแสดงกระดาน

5.3 ฟังก์ชัน new_game()

ฟังก์ชัน `new_game()` ใช้เริ่มเกมใหม่ โดยล้างข้อมูล `mine`, `number` และ `open_cell` แล้วสร้าง Array 2 มิติขนาด 8×8

หลังจากนั้นสุ่มตำแหน่งระเบิดจำนวน 10 ลูก โดยใช้ `random(SIZE)` ถ้าตำแหน่งที่สุ่มได้ยังไม่มีระเบิด จะใส่ระเบิดลงไปและเพิ่มจำนวน `count`

เมื่อวางระเบิดครบแล้ว โปรแกรมจะวนทุกช่องและเรียก `count_mines()` เพื่อคำนวณจำนวนระเบิดรอบช่องนั้น

5.4 ฟังก์ชัน count_mines()

ฟังก์ชัน `count_mines(row, col)` ใช้นับจำนวนระเบิดที่อยู่รอบช่องที่กำหนด โดยตรวจสอบพื้นที่ตั้งแต่แถว `row - 1` ถึง `row + 1` และคอลัมน์ `col - 1` ถึง `col + 1`

ก่อนเข้าถึงข้อมูล โปรแกรมตรวจสอบว่าตำแหน่งนั้นยังอยู่ภายในขอบเขต 0 ถึง 7 เพื่อป้องกันการเข้าถึง Array นอกขอบเขต

5.5 ฟังก์ชัน draw_board()

ฟังก์ชัน `draw_board()` ใช้แสดงกระดาน Minesweeper โดยวนทุกแถวและคอลัมน์

ถ้าช่องยังไม่ถูกเปิดจะแสดงเป็นช่องปิด แต่ถ้าเปิดแล้ว ถ้าช่องนั้นเป็นระเบิดจะแสดงวงกลมแทนระเบิด และถ้าไม่ใช่ระเบิดแต่มีระเบิดอยู่รอบ ๆ จะแสดงตัวเลขจำนวนระเบิด

5.6 ฟังก์ชัน mousePressed()

ฟังก์ชัน `mousePressed()` ทำงานเมื่อผู้เล่นคลิกช่อง โดยคำนวณ `row` และ `col` จากตำแหน่งเมาส์

ถ้าช่องยังไม่ถูกเปิด โปรแกรมจะเปิดช่องนั้นก่อน ถ้าเจอระเบิดจะกำหนด `game_over = True` และเรียก `show_all_mines()` เพื่อแสดงระเบิดทั้งหมด

ถ้าไม่เจอระเบิด โปรแกรมจะเรียก `check_win()` เพื่อตรวจสอบว่าผู้เล่นเปิดช่องปลอดภัยครบหรือยัง

5.7 ฟังก์ชัน show_all_mines()

ฟังก์ชัน `show_all_mines()` ใช้เปิดช่องที่มีระเบิดทั้งหมดเมื่อผู้เล่นแพ้ โดยวนตรวจสอบทุกช่องและตั้งค่า `open_cell[i][j]` เป็น `True` สำหรับช่องที่มีระเบิด

5.8 ฟังก์ชัน check_win()

ฟังก์ชัน `check_win()` ใช้ตรวจสอบเงื่อนไขการชนะ โดยนับจำนวนช่องที่ไม่มีระเบิดทั้งหมดไว้ใน `safe_cells` และนับจำนวนช่องปลอดภัยที่ผู้เล่นเปิดแล้วไว้ใน `opened_cells`

ถ้าจำนวนทั้งสองเท่ากัน แสดงว่าผู้เล่นเปิดช่องปลอดภัยครบทุกช่องแล้ว จึงกำหนด `win = True`

5.9 ฟังก์ชัน keyPressed()

ฟังก์ชัน `keyPressed()` ใช้เริ่มเกมใหม่เมื่อกดปุ่ม R หรือ r โดยเรียก `new_game()` อีกครั้ง

Error ที่พบ

ปัญหาหลักที่ต้องระวังคือการนับระเบิดรอบช่องที่อยู่บริเวณขอบกระดาน เพราะ `row - 1` หรือ `col - 1` อาจมีค่าติดลบ และ `row + 1` หรือ `col + 1` อาจเกินขนาด Array จึงต้องตรวจสอบขอบเขตด้วยเงื่อนไข

```
if i >= 0 and i < SIZE
```

และ

```
if j >= 0 and j < SIZE
```

ก่อนเข้าถึง `mine[i][j]`

อีกส่วนที่ต้องระวังคือการสุ่มระเบิด เพราะอาจสุ่มได้ตำแหน่งเดิมหลายครั้ง จึงต้องตรวจสอบก่อนว่า `mine[r][c] == False` แล้วจึงเพิ่มระเบิด ทำให้สุดท้ายมีระเบิดครบ 10 ลูกโดยไม่วางซ้ำตำแหน่งเดิม

นอกจากนี้ใน `mousePressed()` ต้องตรวจสอบว่า `row` และ `col` อยู่ในขอบเขตของกระดานก่อนเข้าถึงข้อมูล เพื่อป้องกัน `IndexError` เมื่อผู้เล่นคลิกบริเวณนอกกระดาน

สรุปผลการทำงาน

จากการทำ Lab 7 ได้ฝึกการใช้ Array 2 มิติ การสร้างและเรียกใช้ฟังก์ชัน การใช้ลูปเพื่อค้นหาและคำนวณข้อมูล รวมถึงการนำฟังก์ชันหลายฟังก์ชันมาทำงานร่วมกัน

ในส่วนของ Matrix ได้ฝึกการหาค่าสูงสุด ค่าต่ำสุด ค่าเฉลี่ย การค้นหาข้อมูล และการบวก ลบ และคูณ Matrix

ในส่วนของข้อมูลอุณหภูมิ ได้ฝึกการแบ่งข้อมูลเป็นรายสัปดาห์และสร้างฟังก์ชันสำหรับนำข้อมูลไปวิเคราะห์

ส่วน Treasure Hunt ได้ฝึกการใช้ Array 2 มิติร่วมกับการรับ Input จากผู้ใช้ และการควบคุมสถานะของเกมผ่านตัวแปรต่าง ๆ